

Building a Phishing Email Classifier & Threat App

How I designed an end-to-end NLP pipeline with TF-IDF vectorization and Naive Bayes that achieves **98.2% detection accuracy** with sub-3ms latency.

98.2%

TEST ACCURACY

98.7%

PRECISION RATE

0.9994

ROC AUC SCORE

< 3ms

INFERENCE TIME

Python

Scikit-Learn

Multinomial Naive Bayes

TF-IDF N-Grams

Flask REST API

Explainable AI

Why Phishing Detection Is Hard

Over 3.4 billion phishing emails are dispatched daily. Modern spear-phishing campaigns bypass static rule filters while large LLMs are too slow and expensive for high-volume mail gateways.

✗ Why Regex & Rules Fail

- Attackers rotate deceptive URLs, IP links, and lookalike domains constantly.
- Obfuscated HTML formatting and encoded character entities mask malicious payloads.
- High false positive rate blocks legitimate business invoices and IT communications.

✗ Why Large LLMs Aren't Feasible

- **Latency Penalty:** 800ms - 2,500ms per email causes unacceptable mail server backlogs.
- **Extreme Cost:** \$0.005 to \$0.02 per call at 50,000 emails/day = \$9,000+/month.
- **Infrastructure Dependency:** Requires dedicated GPU clusters or external API uptime.

💡 The Engineering Solution: Statistical NLP + Naive Bayes

Deploy a calibrated Multinomial Naive Bayes model powered by sublinear TF-IDF n-grams. Delivers **enterprise-grade 98.2% accuracy**, zero third-party API dependencies, and **sub-3ms CPU inference**.

The Robust NLP Pipeline

Raw email text is notoriously noisy. The preprocessor cleans the payload while systematically preserving critical cybersecurity heuristics.

1 Text Normalization

- **HTML & Entity Stripping:** Unescapes hex entities and strips DOM tags.
- **Semantic Tokenization:** Replaces raw URLs with `token_url` and IP destinations with `token_ip_url`.
- **Security Stopword Filtering:** Preserves domain terms ('verify', 'urgent', 'account') while stripping filler noise.

2 TF-IDF N-Gram Vectorization

- **Unigrams + Bigrams (1, 2):** Captures decisive phrasing like *'account suspended'*, *'wire transfer'*, *'within 24'*.
- **Sublinear Scaling:** Replaces term frequency with $1 + \log(\text{tf})$ to prevent repetitive spam tokens from skewing scores.
- **1,790+ High-Discriminative Features.**

```
# Pipeline Vectorizer Definition
vectorizer = TfidfVectorizer(ngram_range=(1, 2), sublinear_tf=True,
                             min_df=2, max_df=0.95)
X_train_tfidf = vectorizer.fit_transform(cleaned_email_corpus)
```

Multinomial Naive Bayes & Results

Bayes' Theorem provides the optimal balance of mathematical interpretability, probabilistic calibration, and lightning-fast inference speed.

Evaluation on 600 Test Emails

- **True Negatives (TN):** 296 / 300 Legitimate
- **True Positives (TP):** 293 / 300 Phishing
- **False Positives (FP):** Only 4 (1.3%)
- **False Negatives (FN):** 7 (2.3%)
- **Test Accuracy: 98.17% (98.2%)**

Log-Odds Feature Attributions

Top Phishing Weights ($\log P(w|\text{Phish}) / P(w|\text{Ham})$):

- token_url (+5.78)
- immediately (+5.73)
- 24 hours (+5.66)
- unauthorized (+5.45)
- wire transfer (+5.43)

Top Legitimate Weights:

- discussion (-6.33)
- results (-6.20)
- sprint (-6.05)

Zero-GPU Deployment Advantage

Because Multinomial Naive Bayes operates via sparse matrix dot products, inference consumes less than **15MB of RAM** and completes in **< 2.5 milliseconds** on basic CPU instances.

Interactive Flask Web Application

Trained models are only as good as their operational accessibility. I deployed the system with an interactive Flask command center and REST API.

Web Application Features

- **Live Threat Gauge:** Dynamic animated confidence meter (0% to 100%).
- **Explainable AI Highlighter:** Visual color-coded cues for high-risk tokens.
- **Preloaded Attack Presets:** Instant testing of PayPal, CEO Wire Fraud, FedEx, and legitimate sprint updates.

Headless REST API

- POST `/api/scan` enables real-time integration into corporate mail gateways, SIEM pipelines, and SOC triage bots.
- Returns JSON schema with threat severity, confidence scores, and heuristic breakdowns.

Key Takeaway for Machine Learning Engineers

You don't always need billions of parameters. Thoughtful feature engineering, domain preprocessing, and classical probabilistic models can solve real-world cybersecurity problems faster, cheaper, and with superior explainability.